Margin Notes

AI Study Assistant using Retrieval-Augmented Generation (Embeddings + LLM)

Student: Anushay Ayat

Program: Artificial Intelligence and Data Science, SMIT

Assignment: AI Study Assistant using RAG

Technologies: FastAPI, ChromaDB, Groq (gpt-oss-20b), Firebase

GitHub: https://github.com/scientistayat-engineer/ai-study-assistant

1. Introduction

Students who keep their notes as PDF chapters often struggle to find one specific answer quickly. General chatbots can answer, but they may use knowledge from outside the notes and cannot point to a page. This project builds an AI Study Assistant, called Margin Notes, that answers questions only from uploaded Python PDF chapters and shows which file and page each answer came from.

The system uses Retrieval-Augmented Generation (RAG). Instead of asking a language model to answer from memory, the application first retrieves the most relevant passages from the uploaded chapters and then asks a language model (Groq gpt-oss-20b) to write an answer using only those passages.

2. Objectives

- Upload 5 PDF chapters and extract their text.
- Split the text into chunks and generate embeddings for every chunk.
- Save the embeddings locally in ChromaDB.
- Provide a FastAPI web interface where the user asks questions.
- Retrieve the most relevant chunks, send them to the language model, and display the final answer.
- Add login and a polished interface.

3. Background

3.1 Retrieval-Augmented Generation

RAG combines a search step with a generation step. The search step finds passages related to the question, and the generation step turns those passages into a readable answer. This keeps answers tied to the source material and makes it possible to cite sources.

3.2 Embeddings

An embedding is a list of numbers that represents the meaning of a piece of text. Texts with similar meaning have embeddings that are close together. This project uses the all-MiniLM-L6-v2 model, which produces 384 numbers per chunk and runs locally, so no extra API key is needed for embeddings.

3.3 Vector database

ChromaDB stores each chunk with its embedding and metadata (file name and page number). A question is converted to an embedding and compared with the stored ones using cosine distance, and the closest chunks are returned.

4. System Architecture

The web page is served by the FastAPI application itself, which runs on the user's computer (localhost:8000). The service reads and writes ChromaDB, retrieves chunks, and calls the Groq API. Firebase Authentication handles login, and Firestore stores each user's chat history.

Layer	Technology	Role
Frontend	HTML, CSS, JavaScript	Login, chat, quiz, study tools, RAG Inspector

Layer	Technology	Role
Web server	FastAPI (Uvicorn)	Serves the page and the API on localhost
Backend	FastAPI (local; Docker file included)	Upload, retrieval, answering, quiz, study, inspector, evaluation
Vector store	ChromaDB (persistent)	Chunks, embeddings, metadata
Embeddings	all-MiniLM-L6-v2	384-dimension local embeddings
LLM	Groq gpt-oss-20b (Grok from xAI also supported)	Writes answers from retrieved context
Auth and data	Firebase Auth, Firestore	Email and Google login, saved chat history

5. Implementation

5.1 Ingestion pipeline

Each PDF is read page by page with pypdf so that every chunk keeps its page number. The text is cleaned and split into chunks of 900 characters with an overlap of 150 characters, so a sentence cut at a boundary still appears in full in one chunk. ChromaDB creates the embeddings and saves them on disk. Uploading a file again replaces its old chunks.

5.2 Retrieval and answer generation

For each question the backend retrieves the 5 most similar chunks. For follow-up questions, the previous user questions are added to the search text, so a message like "give an example" still finds the right chunks. The retrieved text, the recent chat and the question are sent to the model with an instruction to answer only from the context and to cite sources as [file p.N]. The answer is streamed to the browser word by word.

5.3 Not-in-notes guard

If the best retrieved chunk has a similarity below 0.2 (a configurable setting), the backend does not call the model. It returns a message saying that the answer was not found in the notes. This prevents confident answers that are not supported by the chapters.

5.4 API endpoints

Endpoint	Purpose
POST /api/upload	Upload up to 5 PDFs, extract, chunk, embed and save
GET /api/docs	List indexed PDFs and the number of chunks
POST /api/ask	Retrieve chunks and stream an LLM answer with sources
POST /api/quiz	Generate multiple-choice questions from the chapters
POST /api/study	Chapter summary or flashcards for one PDF
GET /api/inspect	Embedding size, chunks per PDF, sample embeddings
GET /api/eval	Run the retrieval evaluation

5.5 Frontend

The interface is a single page built with HTML, CSS and JavaScript. It has a login screen (email and password, or Google) and four areas: Chat, Quiz, Study and RAG Inspector. Chat supports English, Roman Urdu and Urdu answers, voice input, a copy button for each answer and export of the conversation to PDF through the browser's print dialog. Chat history is saved in Firestore, so it returns after login. A theme switch lets the user choose light, dark or automatic (system) mode, and the choice is remembered.

5.6 Language provider

The backend talks to the language model through an OpenAI-compatible client. The assignment names Grok; because of API credit limits, the final run uses Groq with the openai/gpt-oss-20b model. A setting chooses Groq or Grok (xAI), so the same code works with either provider.

6. Features

- Email and Google login with Firebase Authentication; the backend verifies the token on every request.
- Streaming answers with source chips showing file name and page.
- Chat memory for follow-up questions.
- Answers in English, Roman Urdu or Urdu.
- RAG Inspector showing chunks, embedding dimensions, samples of real embeddings and the similarity scores of the last question.
- Quiz mode with scoring, chapter summaries and flashcards.
- Voice input, copy button, and export to PDF.
- Light, dark or automatic theme, remembered in the browser.

7. Testing and Evaluation

7.1 Automated tests

The file backend/tests/test_api.py runs 16 checks offline, using a fake embedding function and a fake language model so that no API key or model download is needed. All 16 checks pass.

Area	What is checked
Ingestion	5 sample PDFs are indexed; upload re-indexes a file; non-PDF files are rejected
Retrieval	The loops question returns the loops chapter first
Answering	Streaming format, follow-up questions with history
Guard	An off-topic question is refused and returns no sources
Quiz and study	Fenced JSON is parsed; summary and flashcards work; unknown file returns 404
Inspector and evaluation	Embedding size and samples; evaluation runs 10 questions

7.2 Retrieval evaluation

The file backend/data/eval_set.json holds 10 questions with the PDF that should answer each. Running python evaluate.py inside the backend folder (or pressing Run evaluation in the RAG Inspector) reports

the top-1 hit rate, the top-5 hit rate and the mean reciprocal rank (MRR). The results on the 5 sample chapters (10 questions) are below.

Metric	Result	Meaning
Top-1 hit rate	100%	Right PDF ranked first
Top-5 hit rate	100%	Right PDF among the 5 retrieved chunks
MRR	1.0	Average of 1 / rank of the first correct chunk

7.3 Sample questions

The following questions from the assignment were used to test the full system: What are Python variables? Explain the difference between a list and a tuple. What is a for loop? How do functions work in Python? What are conditional statements?

8. Running the Application

The application runs locally: after creating a Python virtual environment and installing requirements.txt, Uvicorn serves the page and the API at http://localhost:8000. Firebase Authentication and Firestore (free Spark plan) provide login and saved chat history, and the Firestore security rules allow each signed-in user to read and write only their own chat history. A Docker file that builds the vector database at image build time is included for hosting the service later, but hosting online was not part of this submission. Setup steps are in docs/SETUP.md.

9. Challenges and Limitations

- The application is not hosted online: it runs only while Uvicorn is running on the local computer, and uploaded PDFs and the vector database stay on that computer.
- Scanned PDFs without a text layer cannot be read; OCR would be needed.
- Chunk size is a trade-off: small chunks are precise but lose context, large chunks add noise.
- Quiz and flashcards depend on the model returning valid JSON; the app shows a clear error and the user can retry.

10. Future Work

- Hosting the application online (frontend on Firebase Hosting, backend in a container).
- OCR for scanned PDFs.
- Separate collections for each user.
- Re-ranking of retrieved chunks.
- A larger evaluation set and tracking of answer quality over time.

11. Conclusion

Margin Notes meets the assignment requirements: it uploads PDFs, extracts and chunks text, creates embeddings, saves them in ChromaDB, retrieves relevant chunks with RAG, and uses a language model (Groq gpt-oss-20b) to produce answers through a FastAPI web interface. The added login, guard against unsupported answers, evaluation, quiz and study tools make it a more complete study tool than a basic question-and-answer demo.

Appendix: Screenshots for submission

- PDF upload
- Embeddings creation (upload confirmation and RAG Inspector)
- Saved vector database (RAG Inspector statistics)
- AI answers to at least 3 different questions
- GitHub repository link